PROJECT 4.1 · UNIT 4 — CONTROL & COMPETITION

Remote Controls (Driver Control)

Student Edition — VEX EXP · C++ in VS Code

Name: _____ **Date:** _____ **Period:** _____

Introduction

Time to **drive your robot by hand**. Chapter 11 was autonomous — sensors decided, no driver. This is the other half: **driver control** (teleoperation), where a human is the decision-maker, moving the robot in real time with the controller's **joysticks** and **buttons**. You'll read a joystick as a number and send it straight to the motors so one stick steers each side (**tank drive**), ignore the center wobble with a **dead zone**, and run a mechanism with a button. Then you'll build and drive a **remote-controlled robot**. Read Chapter 13 in the textbook for the full explanation.

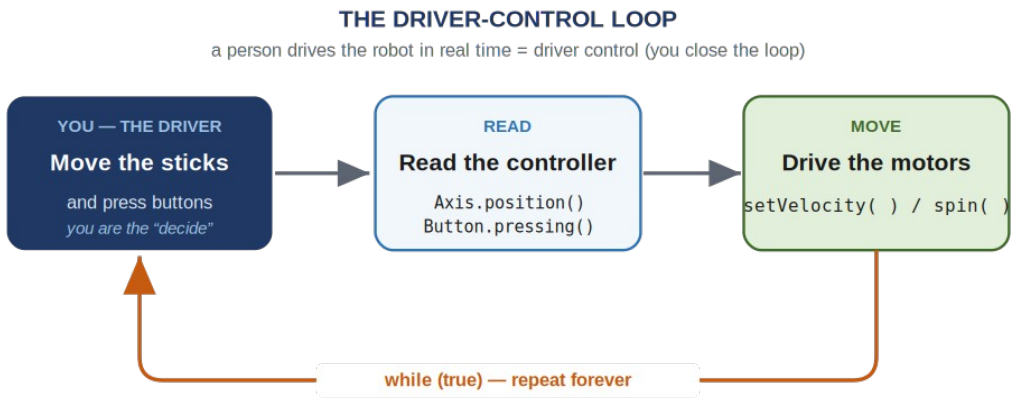
Learning Objectives

- Explain driver control (teleoperation) and contrast it with an autonomous routine — the same loop, but the human is the “decide.”
- Read a joystick axis with position() (a whole number, −100 to 100) and a button with pressing() (true/false).
- Write tank drive — send each stick straight to one side's motor inside a while (true) loop — and add a dead zone so the robot doesn't creep.
- Run a mechanism from buttons with if / else if / else, and build & drive a remote-controlled robot from the kit using the design process.

Key Ideas (quick reference)

- **Driver control is the same loop as automation:** a while (true) loop that goes read → move, over and over. The difference from Chapter 11 is that YOU are the “decide” — the program reads the controller instead of a sensor, and your thumbs choose the action.
- **Reading the controller:** name it once (controller Controller = controller();). A joystick axis: Controller.Axis3.position() returns a whole number −100..100 (full up = 100, full down = −100, centered = 0). A button: Controller.ButtonR1.pressing() returns true or false.
- **Tank drive:** each stick runs one side. Feed Axis3 (left stick) to leftMotor's setVelocity and Axis2 (right stick) to rightMotor's. Both up = forward, both down = reverse, opposite = spin. A negative position with spin(forward) runs the motor backward, so one stick covers both directions. Read it fresh every loop.
- **Dead zone + buttons:** sticks rest at a tiny non-zero value, so ignore small readings: read into int p, then drive only if p > 5 or p < −5, else stop. For a lift or claw, use buttons: if (R1) raise / else if (R2) lower / else stop — the final else is what holds it still when you let go.

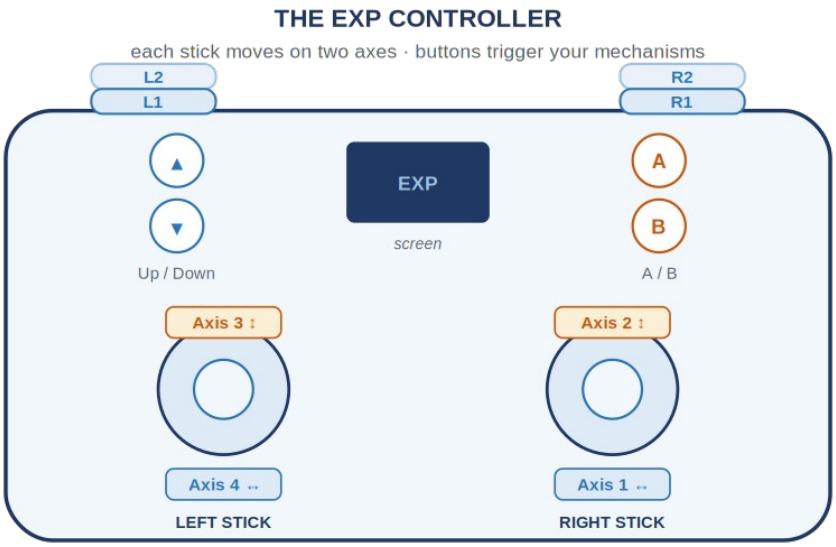
The Driver-Control Loop



In driver control YOU close the loop — your eyes are the sensor.

Driver control is the same while (true) loop as automation — but the driver is the "decide." The program just reads the controller and drives the motors, over and over.

The Controller



Tank drive: left stick (Axis 3) runs the left wheels · right stick (Axis 2) runs the right wheels

The EXP controller has no Left/Right arrows and no X/Y buttons — those are on the V5 controller.

Two sticks give four axes; the buttons are simple pressed / not-pressed switches. For driving you'll use Axis3 (left stick) and Axis2 (right stick).

Name the controller and your motors. The two drive motors are the same leftMotor and rightMotor you set up in Chapter 6 — the right one reversed because it faces the other way:

The device list

```
// name the controller, then your motors:
controller Controller = controller();        // the EXP controller
motor leftMotor  = motor(PORT1);             // left side of the drive
motor rightMotor = motor(PORT6, true);       // right side (reversed)
motor armMotor   = motor(PORT3);             // the lift / claw mechanism
```

Driving: a stick for each side

Ask an axis for its reading with **position()** — a whole number from −100 to 100 (full up = 100, full down = −100, centered = 0). That's a percent, exactly what setVelocity wants, so feed a stick straight to a motor. Each stick runs one side — that's **tank drive**:

Tank drive — left stick runs the left side, right stick the right

```
int main() {
    vexcodeInit();

    while (true) {      // the driver-control loop
        // each stick runs one side (Axis3 = left, Axis2 = right):
        leftMotor.setVelocity(Controller.Axis3.position(), percent);
        rightMotor.setVelocity(Controller.Axis2.position(), percent);
        leftMotor.spin(forward);
        rightMotor.spin(forward);
        wait(20, msec); // pause, then read again
    }
}
```

Both sticks up = straight forward; both down = reverse; one up and one down = spin in place. You never code “reverse”: pulling a stick down makes position() **negative**, and a negative velocity with spin(forward) runs the motor backward. The speed isn't a fixed number — it's read **fresh every loop**, which is what makes driving feel live.

The dead zone. Real sticks rest at a tiny non-zero value, so the robot creeps when untouched. Ignore readings near center: read the axis into a variable, then drive only if it's really moved.

A dead zone for one motor

```
int p = Controller.Axis3.position();        // read the left stick once
if (p > 5 || p < -5) {                       // outside the dead zone?
    leftMotor.setVelocity(p, percent);
    leftMotor.spin(forward);
} else {
    leftMotor.stop();                        // tiny wobble -> hold still
}
```

If p is past ±5 the driver means it, so drive; otherwise hold still. Five is a starting point — widen it to 10 if the robot still twitches.

Buttons: run a mechanism

A lift or claw just goes up or down — a job for a **button**. Ask .pressing() and you get true (held) or false (not). Two buttons run a mechanism both ways:

R1 raises the arm, R2 lowers it

```
if (Controller.ButtonR1.pressing()) {        // R1 held  -> raise
    armMotor.spin(forward);
} else if (Controller.ButtonR2.pressing()) { // R2 held  -> lower
    armMotor.spin(reverse);
} else {
    armMotor.stop();                         // nothing held -> hold position
}
```

It's the same if / else if / else as Chapter 11 — only the condition is now *is the driver pressing this button?* The final else matters: without it, the arm keeps spinning after you let go.

Put It Together: the driver-control loop

Everything lives inside one while (true) loop — read the sticks, drive the wheels, check the buttons, run the arm, then a short wait and around again. That single loop is your whole program:

A complete remote-controlled robot — tank drive + a button-controlled arm

```
int main() {
    vexcodeInit();

    while (true) {
        // --- drive: each stick runs one side ---
        leftMotor.setVelocity(Controller.Axis3.position(), percent);
        rightMotor.setVelocity(Controller.Axis2.position(), percent);
        leftMotor.spin(forward);
        rightMotor.spin(forward);

        // --- arm: R1 raises, R2 lowers ---
        if (Controller.ButtonR1.pressing()) {
            armMotor.spin(forward);
        } else if (Controller.ButtonR2.pressing()) {
            armMotor.spin(reverse);
        } else {
            armMotor.stop();
        }

        wait(20, msec);
    }
}
```

One More Step: Drive the Whole Base as One Thing

Tank-driving each side by hand is perfect for a *driver*. But the run-by-itself moves your Chapter 14 Sumo robot needs — "drive forward exactly 24 inches," "turn to face 90°" — want a cleaner tool: a **smart drivetrain**. Bundle the wheels into one object and give it distances and headings. Because the EXP Brain has a **built-in inertial sensor**, that drivetrain can turn to an exact angle by feel — not by guessing at a timer. Three steps build it: bundle each side into a motor_group, name the Brain's built-in inertial, then combine them into a smartdrive.

Setup — once, near the top with your other devices

```
// bundle each side into a group (add more motors later):
motor_group leftSide  = motor_group(leftMotor);
motor_group rightSide = motor_group(rightMotor);

// the EXP Brain's built-in inertial (the one inside):
inertial DriveIMU = inertial();

// combine into one smart drivetrain that knows heading
// (set wheel travel, track width, wheel base for YOUR bot):
smartdrive Drivetrain = smartdrive(
    leftSide, rightSide, DriveIMU,
    319.19, 250, 200, mm, 1);
```

An inertial sensor has to find "level" before it can measure a turn, so **calibrate it once at the very start** of main() and wait for it to finish. (VEXcode adds this step for you; when you write the code yourself, you add it yourself.)

Calibrate first — at the top of main(), before you drive

```
int main() {
    vexcodeInit();

    // calibrate the inertial first -- hold the robot still:
```

```
    DriveIMU.calibrate();
    while (DriveIMU.isCalibrating()) wait(50, msec);

    // now drive by distance and turn by heading:
    Drivetrain.driveFor(forward, 24, inches);
    Drivetrain.turnToHeading(90, degrees);
}
```

Two new commands do the work: **driveFor** rolls a measured distance, and **turnToHeading** turns until the inertial reads the heading you asked for — the same pair of skills from your Chapter 11 routine, only now the robot measures the turn itself. You'll reuse this exact drivetrain for the Sumo auto-hunt in Chapter 14.

This is the team's "drive" object, early. That one Drivetrain you command by distance and heading is exactly the idea behind the V5 team's SpartanDrive drive object: driveFor(forward, 24, inches) becomes drive.drive_distance(24), and turnToHeading(90, degrees) becomes drive.turn_to_angle(90) — turning to that absolute heading. Same move, new name; the full map is in Appendix B (EXP → V5 Team Bridge).

Build Constraints

Build constraints (still in force). (1) Build your robot from the classroom PLTW Gateway kit only — its bundled motors and controller are fair game; the separate competition team's V5 parts are off-limits. (2) The whole robot must fit one IKEA Kallax cubby — a 33 × 33 cm (13 × 13 in) opening, about 38 cm (15 in) deep. The 33 × 33 face is the binding limit, so design your drive base and lift to pack into that square.

Safety

- A driven robot moves the instant you touch a stick. Give it room to move, and keep hands, hair, and cables clear of wheels, arms, and gears.
- Unplug the USB-C cable before you drive, and drive on the floor or a clear table where the robot can't fall.
- Drive at a speed you can control — start slow, and be ready to drop the sticks to stop.
- Power the Brain down with the X button when you are done, and store the robot in its cubby.

Equipment & Materials

- Your computer with VS Code and the VEX extension, and a USB-C cable
- The PLTW Gateway EXP kit: Brain, charged battery, the controller, two drive motors, and a motor for your mechanism
- Your engineering notebook for sketches, pseudocode, and a daily journal
- This project worksheet (design brief, controls map, and reflection)

Procedure

1. **Define the problem.** Read your task, then fill in the design brief (Worksheet 1): what the robot must do and every constraint — kit only, fits the cubby, which mechanism.
2. **Sketch and choose.** Each teammate sketches a design. Compare them against your criteria and pick the best one to build.

3. **Build the robot.** Construct a sturdy tank-drive base and your mechanism from the kit so the whole thing fits the cubby. Mount both drive motors and the mechanism motor, and wire them to the Brain.
4. **Map your controls, then code.** Fill in the controls map (Worksheet 2) and the pseudocode (Worksheet 3) first. Then translate to C++: name your devices, set up the while (true) loop, drive with the sticks, and add the button if / else if / else.
5. **Test and refine — a little at a time.** Download, unplug, and drive. Fix one thing at a time: add a dead zone if it creeps, widen it if it still twitches, flip a motor if a side runs the wrong way.
6. **Evaluate.** Judge how well it drives, keep your daily journal (Worksheet 4), and answer the conclusion questions — including which was harder, driving or automating, and what you'd hand back to the robot.

Worksheet 1 — Design Brief

Read your task and define the problem before you build.

Section	Your answer
Client / who needs it	
Problem statement — what is the need?	
What it must do (design statement)	
Constraints (kit only, fits cubby)	
Mechanism (besides driving)	

Worksheet 2 — Controls Map

Decide what each stick and button will do **before** you write code. Fill in the middle column; the code on the right is what you'll call.

Control	What it does on the robot	Code (axis or button)
Left stick (up/down)		Controller.Axis3.position()
Right stick (up/down)		Controller.Axis2.position()
Button R1		Controller.ButtonR1.pressing()
Button R2		Controller.ButtonR2.pressing()

Worksheet 3 — Plan the Code

Write the driver-control loop in plain language — the drive lines and the button decision — **before** you write C++.

Pseudocode

```
/*
  while (true)
  {
      drive:
        left  motor speed = _____________________    (left stick)
        right motor speed = _____________________    (right stick)

      mechanism:
        if ( ________________________________ )    // a button
        {
            ________________________________
        }
        else if ( _____________________________ )    // another button
        {
            ________________________________
        }
        else
        {
            ________________________________
        }
  }
*/
```

Worksheet 4 — Daily Journal

A couple of sentences per day — especially what got in your way and how you fixed it.

Day	What we did / obstacles we hit and how we solved them
1	
2	
3	

Conclusion Questions

Answer in complete sentences.

1. What is driver control (teleoperation), and how is it different from the autonomous program you wrote in Chapter 11?
2. Which was harder to create — driving the robot by hand or an autonomous routine? Explain why.
3. If you made this robot autonomous instead, what sensor(s) would it need, and what would each one do?
4. Describe two malfunctions your team had to troubleshoot, and how you overcame them.
5. Describe two responsibilities you held during this project.

6. Explain a dead zone: what problem does it solve, and how does your code create one?